Postman Guide: Error Handling & Negative Testing

This guide focuses on testing **how APIs behave under failure conditions**, including **partial failures, timeouts, retries, malformed payloads, and missing headers**. Proper negative testing ensures robustness and improves system reliability.

---

1. Key Concepts

Negative Testing

- Verifies that the API **handles invalid or unexpected input gracefully**.
- Helps uncover bugs that normal testing may miss.
- Examples:
    - Malformed JSON payloads
    - Missing required headers
    - Unauthorized requests
    - Invalid query parameters

---

2. Partial Failures

- Some APIs may process part of a request successfully while failing others.
- Example: Batch request to create multiple users → 2 succeed, 1 fails

Testing Approach:

- Send batch requests with a mix of valid and invalid data
- Validate:
    - Response status codes for each item
    - Error messages are clear and actionable
    - System state remains consistent

Postman Example:

```
// Post-response script to check partial failure
const res = pm.response.json();

res.forEach(item => {
  if(item.status === "failed") {
    console.log("Failed item:", item.id, "-", item.error);
  } else {
    console.log("Successful item:", item.id);
  }
});
```

---

3. Timeouts

- Occurs when the API takes longer than expected to respond.
- Can happen due to heavy computation, network latency, or server issues.

Testing Approach:

- Use Postman **request timeout settings**
- Validate:
    - API returns meaningful error (504 Gateway Timeout or custom error)
    - Client handles timeouts gracefully

- Retry logic is triggered if applicable

---

4. Retry Logic

- Retry is needed when transient errors occur (timeouts, 500 errors).

Best Practices:

- Define **max retry attempts**
- Use **exponential backoff** to avoid overwhelming the server
- Test both **successful retry** and **failure after max retries**

Example (Conceptual in Postman):

```
let maxRetries = 3;
let attempt = pm.environment.get("retryAttempt") || 0;

if(attempt < maxRetries) {
   attempt++;
   pm.environment.set("retryAttempt", attempt);
   console.log(`Retry attempt: ${attempt}`);
   // Logic to resend request or notify automation script
} else {
   console.log("Max retry attempts reached.");
   pm.environment.set("retryAttempt", 0);
}
```

---

5. Malformed Payloads

- Test invalid or incomplete request bodies.

Examples:

- Missing required fields
- Incorrect data types
- Invalid JSON format

Postman Example:

```
// Invalid JSON (missing closing brace)
{
  "username": "testUser",
  "email": "test@example.com"
```

Testing Approach:

- Expect 400 Bad Request or API-specific validation error
- Validate **error messages are descriptive**

---

6. Missing Headers

- Test requests with required headers missing (e.g., Authorization, Content-Type).

Testing Approach:

- Remove required headers in Postman
- Validate:
    - Correct HTTP status returned (401 Unauthorized, 415 Unsupported Media Type)
    - Clear error messages

Example:

```
// Tests tab to validate missing header response
pm.test("Check missing Authorization header", function () {
```

```
    pm.response.to.have.status(401);
    pm.expect(pm.response.json().error).to.include("Authorization required");
});
```

---

7. Best Practices for Error Handling Testing

- Always **log errors and unexpected responses**
- Use **environment variables** for dynamic retry or test flows
- Combine negative testing with **positive testing** to ensure robustness
- Cover edge cases, such as **empty payloads, extra fields, and invalid formats**